

Universidade do Minho
Escola de Engenharia

Caravela

Uma linguagem para descrever cenas 3D

Engenharia Gramatical · Projeto Parte I
MEI 2026/27

Rafaela Antunes (PG62934) · Tomás Pinto (PG64038) · Nome Apelido (aXXXXX)

Primeiro: o que é o Nau?

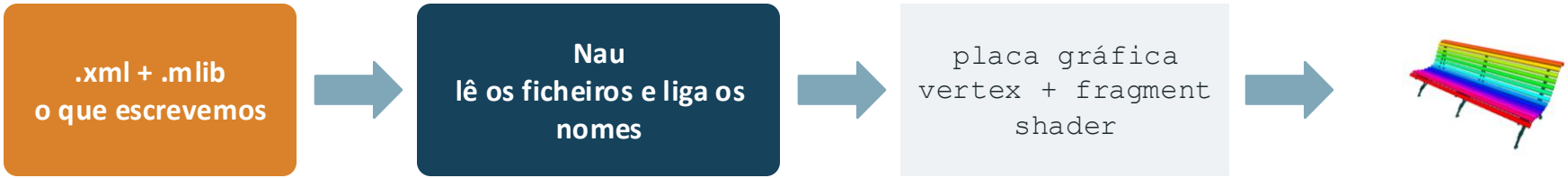
- Motor gráfico da UMinho, usado em Visualização em Tempo Real (VTR).
- Não se programa: descreve-se a cena em ficheiros de texto.
  - **Projeto** `.xml`: objetos, câmara, luzes;
  - **Materiais** `.mlib`: cor e brilho;
  - **Shaders** `.vert` `.frag`: correm na placa gráfica;



Uma chaleira, uma câmara e uma luz (aula 2 de VTR)

Comparação: é como uma página web, onde o HTML, o CSS e o JavaScript estão em ficheiros diferentes e só se ligam por nomes.

Como funciona o Nau?



- **Para desenhar,** usa-se um passe (pass): uma ordem que diz que cena desenhar, com que câmara e com que luzes.
- **O material** de cada objeto indica o shader que o desenha.
- **Os valores** (cor, direção da luz, matrizes) chegam aos shaders como uniforms, ligados só pelo nome.
- **O vertex shader** corre uma vez por vértice (onde aparece no ecrã) e o fragment shader uma vez por píxel (que cor tem).
- **Se o nome de um uniform estiver errado,** o Nau desenha na mesma com um valor por omissão: a cor a zero, ou a textura errada.

O problema: erros que passam

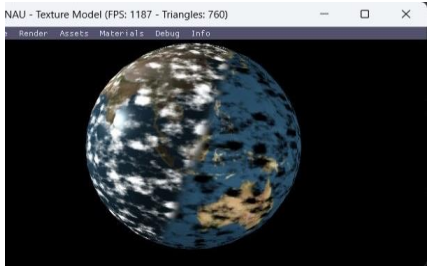
365 linhas,

em 5 ficheiros, para a Terra das fotos.

- Os ficheiros ligam-se só por nomes.
- Um uniform com o nome errado não pára o Nau: a cena abre na mesma, mas com o aspeto errado.
- O aviso só aparece no log, no meio de muitas outras mensagens.



Certo: texNight — à noite, aparece o mapa da noite

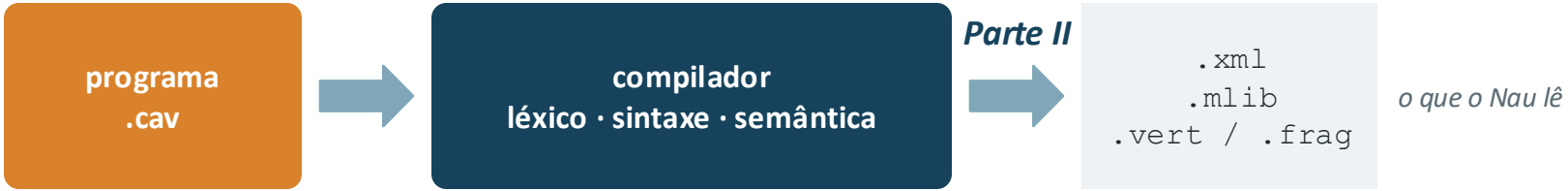


Errado: texNi — à noite aparece o mapa do dia

```
<!-- textureModel.mlib -->
<valueof uniform="texNi" type="TEXTURE_BINDING"
         component="UNIT" id="2" />

// texture.frag
uniform sampler2D texNight;  // unidade 0
```

A nossa ideia: a Caravela



- **Escrever menos:** cada coisa é declarada uma vez, e os shaders serão gerados na Parte II.
- **Erros antes do Nau:** verificar se cada nome existe e tem o tipo certo.
- **Cenas com lógica:** variáveis, ciclos, condições e templates.
- **Âmbito:** iluminação e texturas (aulas 2 e 3 de VTR), não o Nau todo.
- **Para quem:** alunos de VTR que querem saber dos erros antes de abrir o Nau, em vez de os procurar no log depois de abrir.

Language design: as decisões

```
material textured {
    lighting phong { shininess 128; }
    layer day   earth;
    layer night eNight;
}

scene templo {
    for i in 0..n {
        let ang = i * 360 / n;
        pilar(3, azul) { rotate ang around (0,1,0); }
    }
}

pass render { draw templo; camera cam; }
```

Decisões

- **Efeitos em vez de shaders:** o material diz o que se quer ver (`lighting`, `layer`), o compilador escolhe o shader
- **Tudo declarado com nome, num só ficheiro:** dá para ter tabela de símbolos
- **Blocos aninhados:** cena → objeto → transformações
- **let, for, if, template:** resolvidos em compilação, porque o XML só aceita valores fixos
- **Âmbito estático:** um nome de dentro esconde o de fora

O programa tem a mesma forma em árvore que a própria cena

Exemplo 1: o projeto da aula 2

Original (XML do Nau)

```
<camera name="MainCamera">
  <viewport name="MainViewport" />
  <POSITION x="1.46" y="1.05" z="1.75" />
  <LOOK_AT_POINT x="0" y="0" z="0" />
</camera>
...
<pass class="default" name="pass1">
  <scenes><scene name="teapot_s" /></scenes>
  <camera name="MainCamera" />
  <lights><light name="Sun" /></lights>
  <injectionMaps><map toMaterial="*">
    <shader fromLibrary="simple"
            fromMaterial="simple" />
  </map></injectionMaps>
</pass>
```

Caravela

```
camera MainCamera {
    position (1.46, 1.05, 1.75);
    look_at  (0, 0, 0);
    near     0.01;
}
light Sun directional { direction (2, 1, 2); }

pass pass1 {
    draw   teapot_s;
    camera MainCamera;
    lights Sun;
    lighting per_vertex for all;
}
```

175 linhas → 28 linhas e 4 ficheiros → 1 ficheiro. A última linha do passe substitui o shader e todos os <valueof> do .mlib.

Exemplo 2: template, ciclo e condição

```
let n    = 8;
let raio = 4;

template pilar(h: num, mat: material) {
    box p : mat {
        scale     (0.5, h, 0.5);
        translate (0, h / 2, 0);
    }
}

scene templo {
    for i in 0..n {
        let ang = i * 360 / n;
        if i % 2 == 0 {
            pilar(3, azul)   { rotate ang around (0,1,0); }
        } else {
            pilar(2, branco) { rotate ang around (0,1,0); }
        }
    }
}
```

8 pilares em círculo, que no XML seriam 8 objetos escritos à mão, com as posições calculadas à parte.

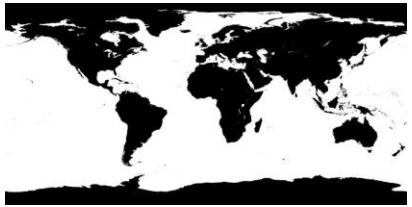
A Caravela apanha:

- `pilar(3, azl)` não existe
- `pilar(azul, 3)` troca os tipos
- `ang` fora do ciclo
- `draw templo, cam` (não é cena)
- `shininess` sem phong

Exemplo 3: texturas da Terra (aula 3)



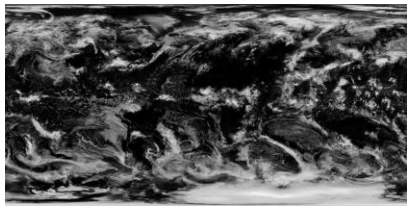
day: cor de dia



specular: onde brilha (mar)



night: luzes das cidades



clouds: nuvens (ruído)

```
material textured {
    lighting phong { shininess 128; }
    layer day      earth;
    layer specular spec;
    layer night    eNight;
    layer clouds   noise speed 0.0001;
}
```

- No Nau, cada imagem tem um número (unidade de textura) escrito em dois sítios. Trocar dois números = imagens trocadas, sem aviso.
- Na Caravela diz-se só o papel de cada imagem. As unidades **0, 1, 2** são calculadas por um atributo.

A gramática: visão geral

```
start: project? decl*

decl: let_decl | texture_decl | material_decl
    | scene_decl | template_decl | camera_decl
    | light_decl | background_decl | pass_decl

material_decl: "material" ID "{" mat_item* "}"
mat_item: diffuse | mat_lighting | layer
layer: "layer" layer_role layer_source ";"
layer_role: DAY | SPECULAR | NIGHT | CLOUDS

scene_item: object | template_call | let_decl
          | scene_for | scene_if
scene_for: "for" ID "in" num _RANGE num "{" scene_item* "}"
scene_if: "if" cond scene_block ("else" (scene_block |
scene_if))?
scene_block: "{" scene_item* "}"

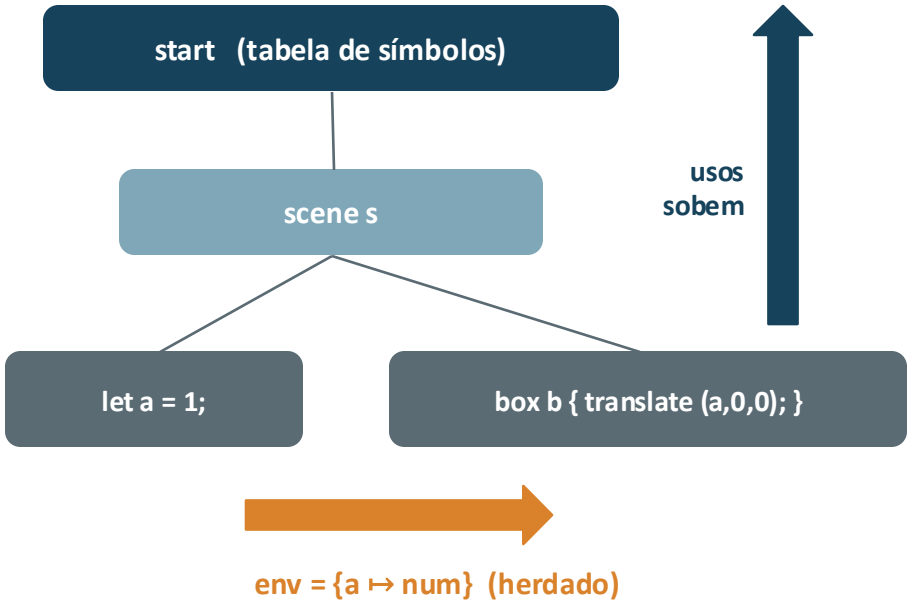
// expressões
num: num ADD term | num SUB term | term
term: term MUL_OP factor | factor
factor: NUMBER | ID | SUB factor | "(" num ")"
```

Organizada por conceitos

- Texturas e materiais
- Cenas e templates
- Câmara, luz e fundo
- Passes
- **Valores:** `num`, `vec3`, `color`, `cond`, usados por todos
- **Léxico:** `ID`, `NUMBER`, `STRING`, operadores, comentários `//` e `/* */`

LALR(1) · 71 não-terminais · 76 terminais · 58 palavras reservadas · 0 conflitos

Gramática de atributos (LarkAG)



- Cada nome usado gera um **uso**: (nome, tipo esperado)
- **Nomes locais** (let numa cena, for, template): descem no `env`, da esquerda para a direita
- **Nomes globais** podem vir depois: os usos sobem até `start` e são comparados com a tabela
- Uma só visita: a gramática é L-atribuída

$$\texttt{sceneitem.env} = \texttt{sceneitems}_0\texttt{.env} \oplus \texttt{sceneitems}_1\texttt{.locais}$$

108 atributos (102 sintetizados, 6 herdados) · 225 regras de avaliação · 25 condições de contexto

O que a semântica apanha

- Nome não declarado ou declarado duas vezes
- Tipo errado (`draw` só aceita cenas)
- Argumentos de um template com tipos trocados
- Ciclos entre `let` e templates recursivos
- Materiais incoerentes (`shininess` só em phong)
- Calcula as unidades de textura de cada material

Ficou de fora: valores válidos (near > 0), porque obrigava a avaliar as expressões.

```
$ python -m caravela exemplos/3_templo.cav
exemplos/3_templo.cav: programa correto

# pilar(3, azul) -> pilar(azul, 3)
$ python -m caravela templo_erro.cav
erro semântico: Argumentos do template
com tipo errado

# draw templo, chao -> draw templo, cam
$ python -m caravela templo_erro2.cav
erro semântico: Tipo errado
```

Parte II: a gramática anotada

```
start: project? decl*                  // V  tabela de símbolos, avisos de "não usado"
let_decl: "let" ID "=" value ";"       // I  guarda o valor no ambiente
num: num ADD term | ...                // I  avalia: i * 360 / n  ->  0, 45, 90 ...
scene_for: "for" ID "in" num .. num    // I  desenrola: uma cópia do corpo por cada i
scene_if: "if" cond scene_block ...    // I  escolhe o ramo já na compilação
template_call: ID "(" args? ")" ...    // I  expande o corpo com os argumentos
object: shape ID (":" ID)? ...         // G  <geometry name="templo_p_3" ...>
material_decl: "material" ID ...       // G  <material> no .mlib + shader GLSL
pass_decl: "pass" ID "{" ... "}"       // G  <pipeline><pass> no .xml
```

- **V Visitor:** percorre a árvore e recolhe informação
- **I Interpreter:** controla a visita: avalia, desenrola os ciclos e expande os templates
- **G Gerador:** um Interpreter que escreve o .xml, o .mlib e os shaders

Qualidade da linguagem

Fator (NB07)	Na Caravela	Limite
Expressividade	conceitos de VTR (lighting phong, layer night); 175 → 28 linhas	condições sem and/or
Documentação	comentários // e /* */	sem documentação estruturada
Unicidade	quase sempre uma só forma de fazer cada coisa	lighting ou material for all no passe
Consistência	declarações: palavra nome { … } ou palavra nome = valor;	background e project não têm nome
Extensibilidade	let, material e template criam nomes novos	sintaxe nova obriga a mudar a gramática
Escalabilidade	for, if e template em vez de repetir: 8 pilares num ciclo	o for só avança de 1 em 1
Fiabilidade	tipos e erros em 3 níveis: léxico, sintaxe e semântica	mensagens sem nome nem linha
Modularidade	template reutiliza partes de uma cena	sem módulos nem import

Demos prioridade à fiabilidade e à expressividade, porque o problema é detetar erros cedo.

Qualidade da gramática

Gramática	Valor
Terminais / não-terminais	76 / 80 (71 escritos)
Produções (unitárias)	174 (59)
Lado direito	média 2,38, máx. 9
Grafo de dependências	130 arestas, grau 1,62
Parser LALR(1)	330 estados, 0 conflitos
Recursividade indireta	só blocos e expressões
Notação	mista: 23 EBNF, 48 BNF

Gramática de atributos	Valor
Atributos	108 (102 SA, 6 IA)
Tipos	88 estrut., 18 atómicos, 2 any
Regras de avaliação	225 (97 cópias)
Condições de contexto	25
Avaliação	L-atribuída, 1 visita

Compreensão: nomes dos conceitos, regras curtas

Manutenção: duas versões da gramática (Lark e LarkAG), um doctest mantém-nas iguais

Reconhecimento: LALR(1) sem conflitos